LSTM

Long Short-Term Memory

Core Idea, Gates, Notation & Full Architecture Explained Visually

Core Idea: The Cell State Is Like a Conveyor Belt

Information can flow along this line almost unchanged.
Gates only add small, carefully-controlled modifications at each step -
this is why LSTMs can preserve information over long sequences.

Part 1: The Core Idea & The Gates | Part 2: The Architecture - How It All Works

PART 1 - The Core Idea & The Gates

1. What Is LSTM?

Long Short-Term Memory (LSTM) is a special kind of RNN, designed specifically to learn long-range dependencies - something vanilla RNNs struggle with due to vanishing gradients. LSTMs were introduced by Hochreiter & Schmidhuber (1997) and have since become one of the most widely used building blocks for sequence modeling.

Every recurrent network has a repeating chain of cells. In a vanilla RNN, this repeating cell is extremely simple - just a single tanh layer. LSTMs use the same chain structure, but the repeating cell is far more sophisticated, containing four interacting components instead of one.

2. The Core Idea: The Cell State

The single most important idea in an LSTM is the **cell state**, C(t) - think of it as a conveyor belt running straight down the entire chain, with only a few minor, carefully controlled linear interactions along the way. Information can ride along this belt largely unchanged, which is exactly what allows it to persist across long sequences.

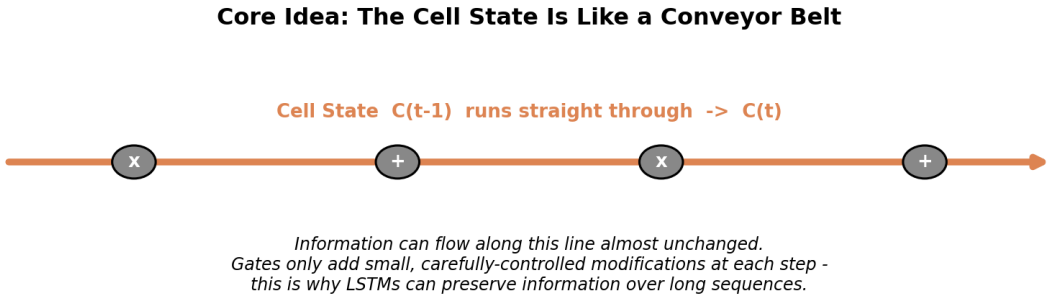

Figure 1: The cell state acts like a conveyor belt - information flows through with only small, regulated changes.

The LSTM does have the ability to remove or add information to this cell state, but it does so very deliberately, through structures called **gates**.

3. LSTM Gates - The Core Concept

A **gate** is a way to optionally let information through. It's composed of a sigmoid neural network layer and a pointwise multiplication operation. The sigmoid layer outputs numbers between 0 and 1, describing how

much of each component should be let through:

- An output of **0** means "let nothing through."
- An output of **1** means "let everything through."
- Values in between allow partial, learned amounts of information to pass.

An LSTM has exactly **three** of these gates, each protecting and controlling the cell state:

- **Forget Gate** - decides what old information to throw away from the cell state.
- **Input Gate** - decides what new information to store in the cell state.
- **Output Gate** - decides what part of the cell state to reveal as the output.

These three gates are the entire reason LSTMs can selectively remember or forget information over very long sequences - each gate learns, through training, exactly when to open, close, or partially open.

PART 2 - The Architecture: The How?

4. Notation - What Are C(t), h(t), x(t), f(t), i(t), o(t)?

Before walking through the mechanics, here is every symbol used in the LSTM cell diagram, all in one place:

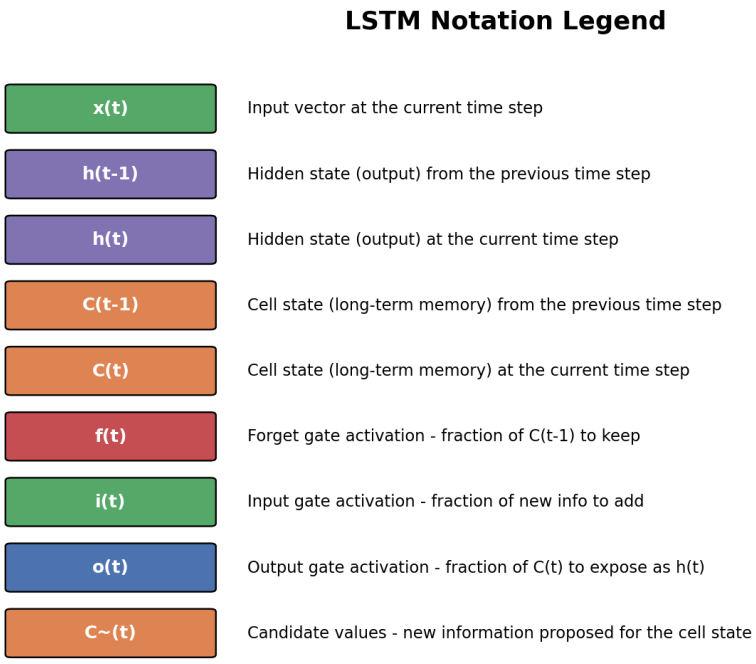

Figure 2: Every variable used inside an LSTM cell, defined in one place.

5. Pointwise Operators & Neural Network Layers

The LSTM diagram uses a small set of repeating visual symbols. Getting comfortable with these makes the full diagram much easier to read:

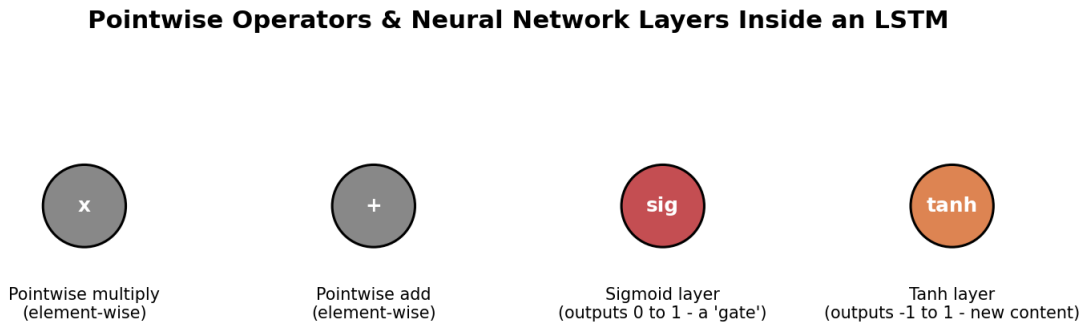

Figure 3: The building-block symbols used throughout the LSTM cell - pointwise operators and neural network layers.

Note the distinction: a **pointwise operator** (x or +) performs a simple element-wise arithmetic operation on two vectors of the same size, with no learned parameters. A **neural network layer** (sigma or tanh) has its own learned weights and bias, and transforms its input into a new vector.

6. The Full LSTM Cell Architecture

Putting it all together, here is the complete LSTM cell for a single time step t. At the bottom, the previous hidden state h(t-1) and the current input x(t) are concatenated and fed into all four internal layers (forget gate, input gate, candidate layer, output gate). At the top, the cell state flows through, is selectively modified, and produces the new cell state C(t) and hidden state h(t).

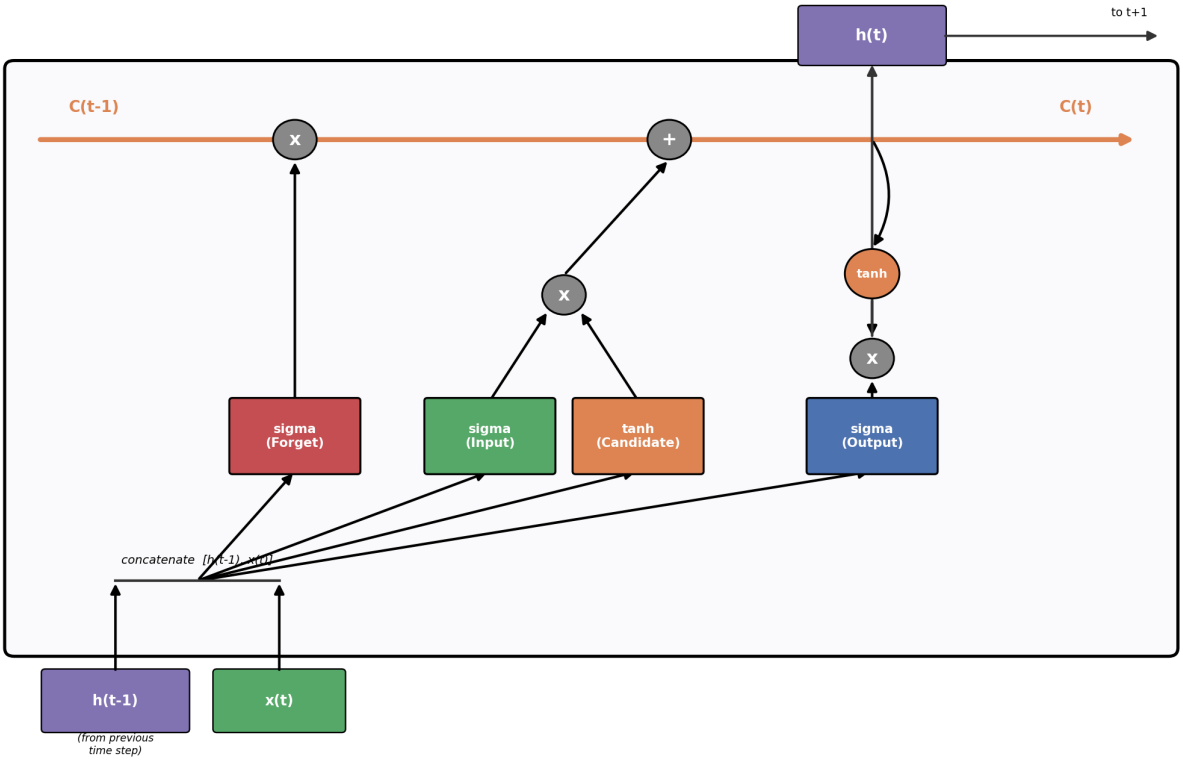

Figure 4: The complete LSTM cell - all four gates/layers acting on the cell state and hidden state.

7. Step-by-Step Walkthrough

Step 1: The Forget Gate

The first step decides what information to throw away from the cell state. This decision is made by the forget gate, a sigmoid layer that looks at h(t-1) and x(t) and outputs a number between 0 and 1 for each number in the cell state C(t-1).

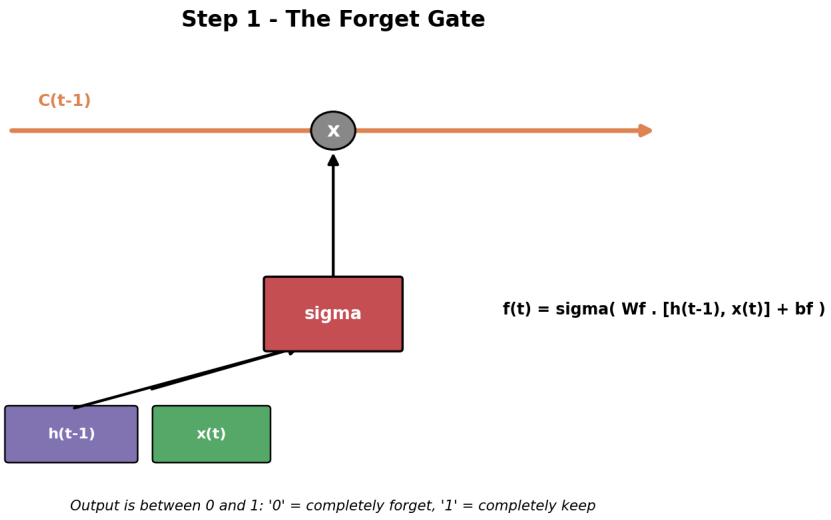

Figure 5: The forget gate decides what fraction of the old cell state to keep.

$$f(t) = \text{sigma}(Wf \cdot [h(t-1), x(t)] + bf)$$

Step 2: The Input Gate & Candidate Values

Next, the LSTM decides what new information to store in the cell state. This has two parts: a sigmoid layer called the **input gate** decides which values to update, and a tanh layer creates a vector of new **candidate values**, C~(t), that could be added to the state.

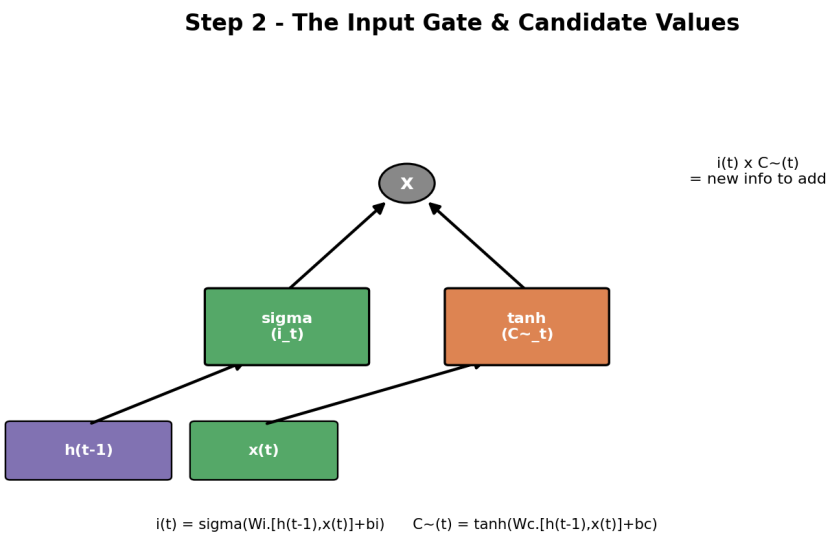

Figure 6: The input gate and candidate layer work together to propose new information.

$$i(t) = \text{sigma}(Wi \cdot [h(t-1), x(t)] + bi)$$

$$C\sim(t) = \tanh(Wc \cdot [h(t-1), x(t)] + bc)$$

Step 3: Updating the Cell State

Now the old cell state C(t-1) is updated into the new cell state C(t). We multiply the old state by f(t), forgetting the things we decided to forget earlier, then add i(t) * C~(t) - the new candidate values, scaled by how much we decided to update each value.

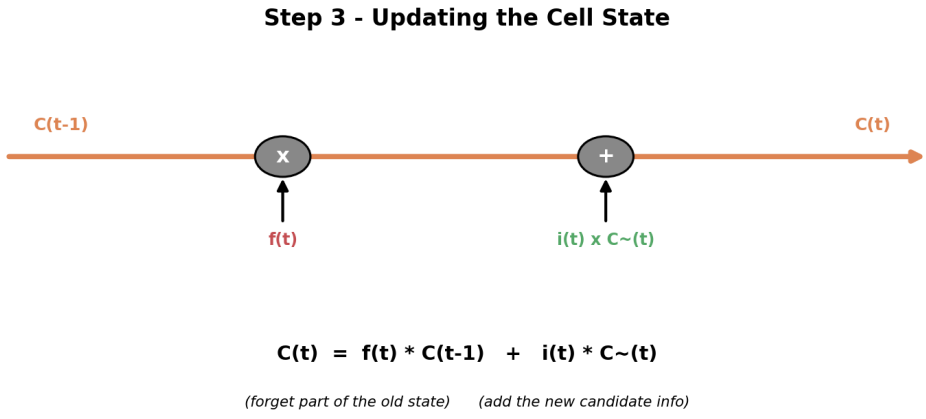

Figure 7: The cell state update combines the 'forget' and 'input' contributions.

$$C(t) = f(t) * C(t-1) + i(t) * C\sim(t)$$

Step 4: The Output Gate

Finally, the LSTM decides what to output. This output is based on the cell state, but filtered. A sigmoid layer (the **output gate**) decides what parts of the cell state to output, then the cell state is pushed through

tanh (to push values between -1 and 1) and multiplied by the output gate, so we only output the parts we decided to.

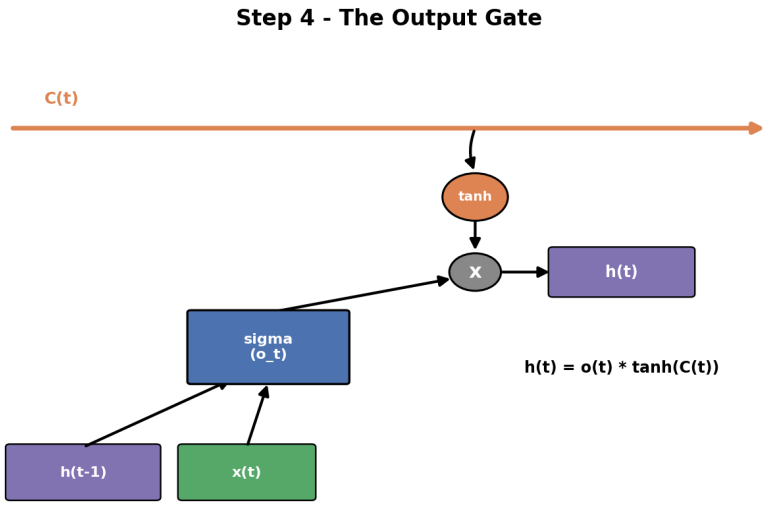

Figure 8: The output gate produces the new hidden state h(t), which is also the cell's output at this time step.

$$o(t) = \text{sigma}(Wo \cdot [h(t-1), x(t)] + bo)$$

$$h(t) = o(t) * \tanh(C(t))$$

8. Summary - All Equations Together

Gate / Value	Equation	Role
Forget gate f(t)	sigma(Wf.[h(t-1),x(t)]+bf)	What to discard from C(t-1)
Input gate i(t)	sigma(Wi.[h(t-1),x(t)]+bi)	What new info to add
Candidate C~(t)	tanh(Wc.[h(t-1),x(t)]+bc)	New candidate content
Cell state C(t)	f(t)*C(t-1) + i(t)*C~(t)	Updated long-term memory
Output gate o(t)	sigma(Wo.[h(t-1),x(t)]+bo)	What part of C(t) to output
Hidden state h(t)	o(t) * tanh(C(t))	Output of this time step

The magic of the LSTM is that all six of these equations are learned jointly through backpropagation through time, so the network itself learns exactly when to forget, when to update, and when to output - without anyone hand-coding those rules.

This is why LSTMs (and their simpler cousin, the GRU) became the standard tool for sequence modeling for many years, until attention-based Transformer architectures took over for tasks needing very long context windows.